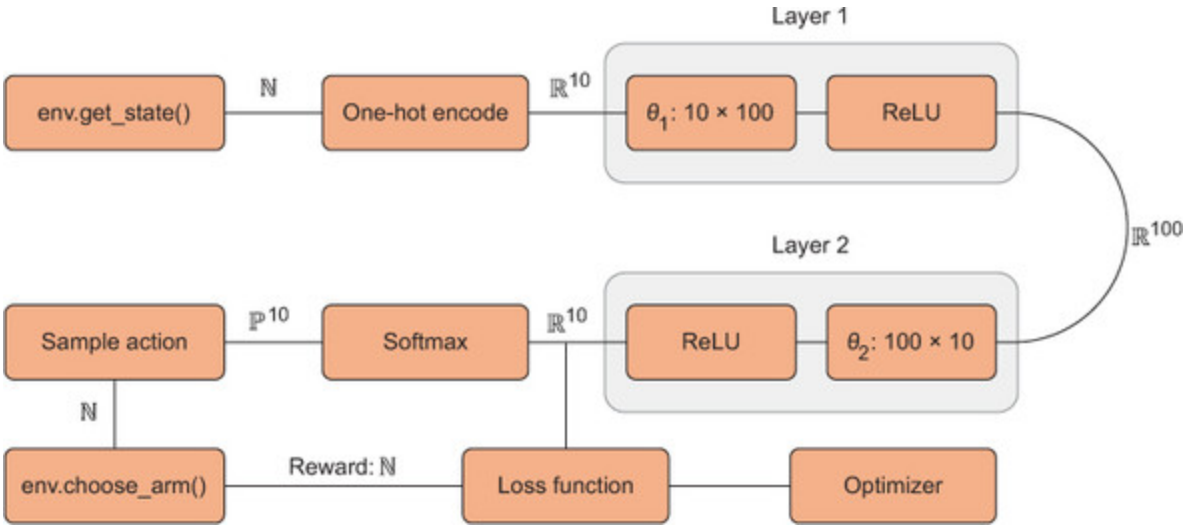


Initially our neural network will produce a random vector such as `[1.4, 50, 4.3, 0.31, 0.43, 11, 121, 90, 8.9, 1.1]` when in state 0. We will run softmax over this vector and sample an action, most likely action 6 (from actions 0 through 9), since that is the biggest number in the example vector. Choosing action 6 will generate a reward of say 8. Then we train our neural network to produce the vector `[1.4, 50, 4.3, 0.31, 0.43, 11, 8, 90, 8.9, 1.1]`, since that is the true reward we received for action 6, leaving the rest of the values unchanged. The next time when the neural network sees state 0, it will produce a reward prediction for action 6 closer to 8. As we continually do this over many states and actions, the neural network will eventually learn to predict accurate rewards for each action given a state. Thus, our algorithm will be able to choose the best action each time, maximizing our rewards.

The following code imports the necessary libraries and sets up some *hyperparameters* (parameters to specify model structure):

```
import numpy as np
import torch

arms = 10
N, D_in, H, D_out = 1, arms, 100, arms
```

In the preceding code, `N` is the batch size, `D_in` is the input dimension, `H` is the hidden dimension, and `D_out` is the output dimension.